# Step 1: Interaction Mechanics & Input Paradigm Mapping

**User Selection Flow:** For this case study, I will be focusing on the user flow of searching for information on Early Voting from the Madison County Elections website.

Here is a link to that specific subpage of the Madison County Elections website: https://www.madisoncountyil.gov/departments/county_clerk/elections/early_voting.php

Currently, the website offers a brief couple of paragraphs about what early voting is and when it first starts (15 days before the election) at the top. Subsequently if a user scrolls down, there is a long list of bolded locations/addresses with unbolded dates and times beneath them during which you can vote early at that location. Each location is separated by a line break from the previous and next ones. It is also offered in a pdf form above the list embedded in the page. Finally, there are three more paragraphs below the list of early voting locations that answer the questions: “Is the Ballot the same?”, “What kind of election equipment does Madison County use for Early Voting?”, and “I voted early. Can i cancel my vote?” all separated by line breaks as well.

I think some of the additional information, like when early voting starts and the three FAQs at the bottom, are helpful, but I found the early voting locations list to be lacking as it’s hard to manually parse.

I would posit that an improved user flow would consist of the following steps:

1. Location Selection: The user selects a location from a dropdown menu to search for nearby early voting locations.
2. Search Submission: The user clicks a button labeled “Find Information” to initiate the search.
3. System Processing: The interface displays a loading indicator while the system processes the request.
4. Results Display: The interface displays the relevant early voting location(s), including their addresses and available voting dates and times.
5. Error Handling: If the user's input is invalid or the search fails, the interface displays an appropriate error message with instructions for correcting the issue or retrying the search.

**Input Mechanic Analysis:** The proposed early voting location search interface will support multiple input paradigms to accommodate users accessing the website through different devices and assistive technologies.

1. Mouse/Pointing Device Input

Users navigating the website with a mouse will be able to click the dropdown menu to view the available early voting locations and select their desired location. After making their selection, users can click the "Find Information" button to initiate the search.

Interactive elements will provide visual feedback when the mouse hovers over them, such as a change in background color or border appearance. This should help communicate to users which elements are interactive and which are not. For example, the “Find Information” button will light up in a darker color once a selection has been made from the drop down, inviting the user to click once their search is primed and ready.

2. Touchscreen Input

Users accessing the website through a smartphone or tablet will be able to tap the dropdown menu to select an early voting location and tap the "Find Information" button to submit their selection.

All interactive elements will have sufficiently large touch targets (at least 44 × 44 pixels), with appropriate spacing between adjacent controls to reduce accidental selections. The interface will not require complex gestures such as swiping, dragging, or pinching to complete the search.

3. Keyboard Input

Users who cannot or prefer not to use a mouse will be able to navigate the interface using these keys on their keyboards:

-Tab: Moves focus between interactive elements, such as the location dropdown and search button.

-Shift + Tab: Moves focus backward through interactive elements.

-Enter: Activates the search button or submits the form.

-Arrow Keys: Allow users to navigate options within the native dropdown menu of early voting locations.

-Space: Activates focused buttons and may open the native dropdown menu, depending on the browser.

Each interactive element will have a clearly visible focus indicator to help keyboard users identify their current position within the interface (unlike the current main page of the Madison County Elections website whose focus indicators are lackluster at best, especially on darker bordered page elements). The keyboard focus order will follow the logical sequence of selecting a location and submitting the search.

4. Screen Reader and Assistive Technology Input

The interface will use semantic HTML elements and accessible labels to support users who rely on screen readers.

The location dropdown will have a descriptive label identifying its purpose, and the search button will have a meaningful, accessible name, such as "Find Early Voting Information."

For example, a screen reader may announce "Searching for early voting information" while the system processes the request, followed by "Voting information loaded for (selected location)" once the results are available.

**Fitts' Law Target Sizing:** Fitts' Law describes the relationship between the size of an interactive target, its distance from the user's starting position, and the time required to select it. Larger targets generally require less precision and are easier to interact with, particularly on touchscreen devices.

For the redesigned early voting location search interface, interactive elements will be sized and spaced to improve usability and minimize accidental input errors.

The following dimensions are proposed for the interactive prototype:

| Interactive Element | Proposed Target Size | Minimum Spacing |
|---|---|---|
| Location Dropdown Menu | 280 x 48px | 12px |
| "Find Information" Button | 180 x 48px | 12px |
| Clear/Reset Button (if included) | 100 x 48px | 12px |
| Navigation Links (if included) | At least 44 x 44px | 8px |

All primary interactive elements will meet or exceed a minimum touch target size of 44 × 44 pixels. The additional spacing between controls will help reduce accidental selections, especially for users accessing the website through mobile devices like phones and tablets.

Applying Fitts' Law

Fitts' Law can be expressed using the following equation: $MT = a + b \times \log_2(1 + D/W)$, where…

-MT = Movement time
-D = Distance from the starting position to the target
-W = Width of the target along the direction of movement
-a and b = Empirically determined constants

For example, assuming a pointer must travel 300 pixels to reach a button:
A 24px-wide target has an index of difficulty of approximately 3.75 bits, while a 48px-wide target has an index of difficulty of approximately 2.86 bits.

Increasing the target width from 24px to 48px reduces the index of difficulty by approximately 24%, indicating that, with all other variables equal, the larger target should be easier to acquire.

This calculation demonstrates why the redesigned interface will use larger interactive targets rather than small text links or tightly spaced controls. It is so the user will be able to interact with the Madison County Elections website more quickly and easily.

Additionally, the location dropdown and "Find Information" button will be arranged vertically, with consistent spacing between them. This layout minimizes unnecessary pointer movement and makes the interface easier to use on both desktop and mobile (touch-based) devices.

# Step 2: High-Fidelity Prototyping & Micro-Interaction Design

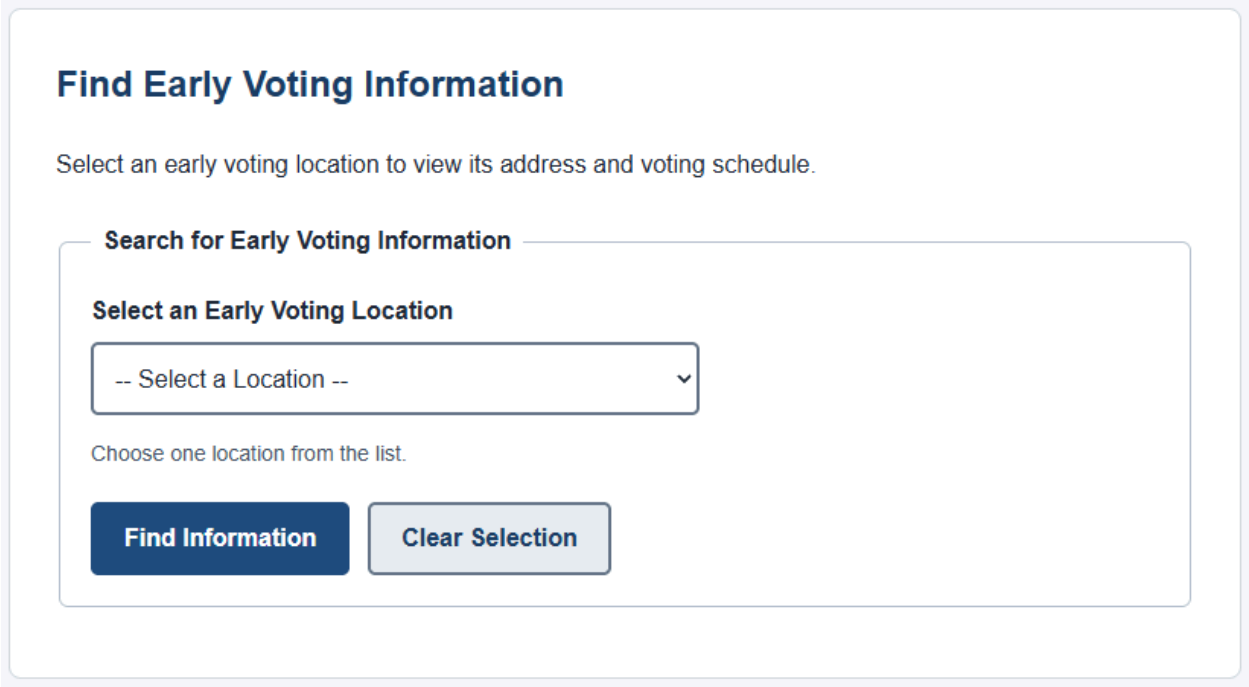
**Interactive Prototype Creation**: The files for the interactive prototype can be found in the “early-voting-prototype” folder, located in the top level directory of this GitHub repository. If you would like to launch it using netlify, please click this link: https://eclectic-lebkuchen-dc2c7d.netlify.app

**State Transitions Definitions**: For the redesigned early voting information interface, I implemented seven distinct interaction states to ensure that users receive appropriate visual and functional feedback when selecting and searching for an early voting location. These states include Default/Resting, Hover/Pointer-Over, Focus, Active/Pressed, Disabled, Loading/System Processing, and Error/Validation Alert.

1. Default/Resting State

The default state represents the interface before the user interacts with it. The location dropdown appears with a white background and dark border, displaying the placeholder text "-- Select a Location --". The "Find Information" button appears with a blue background and white text, while the "Clear Selection" button has a lighter background.

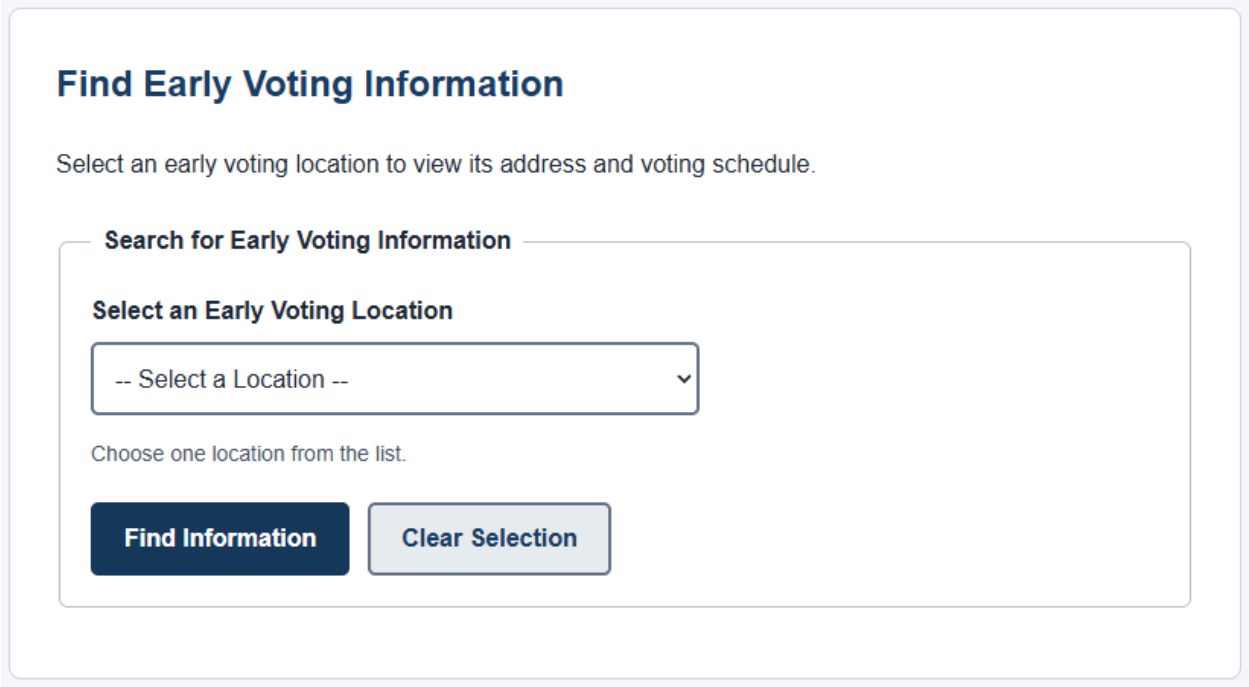
Both buttons and the dropdown are enabled, allowing users to begin interacting with the interface immediately.



2. Hover/Pointer-Over State

When a user moves their mouse over an interactive element, its appearance changes to indicate that it can be selected.

For example, hovering over the "Find Information" button changes its background to a darker blue, while hovering over the dropdown changes its border color and background. These changes are implemented using the CSS :hover pseudo-class, with a 200ms transition to provide smooth visual feedback.

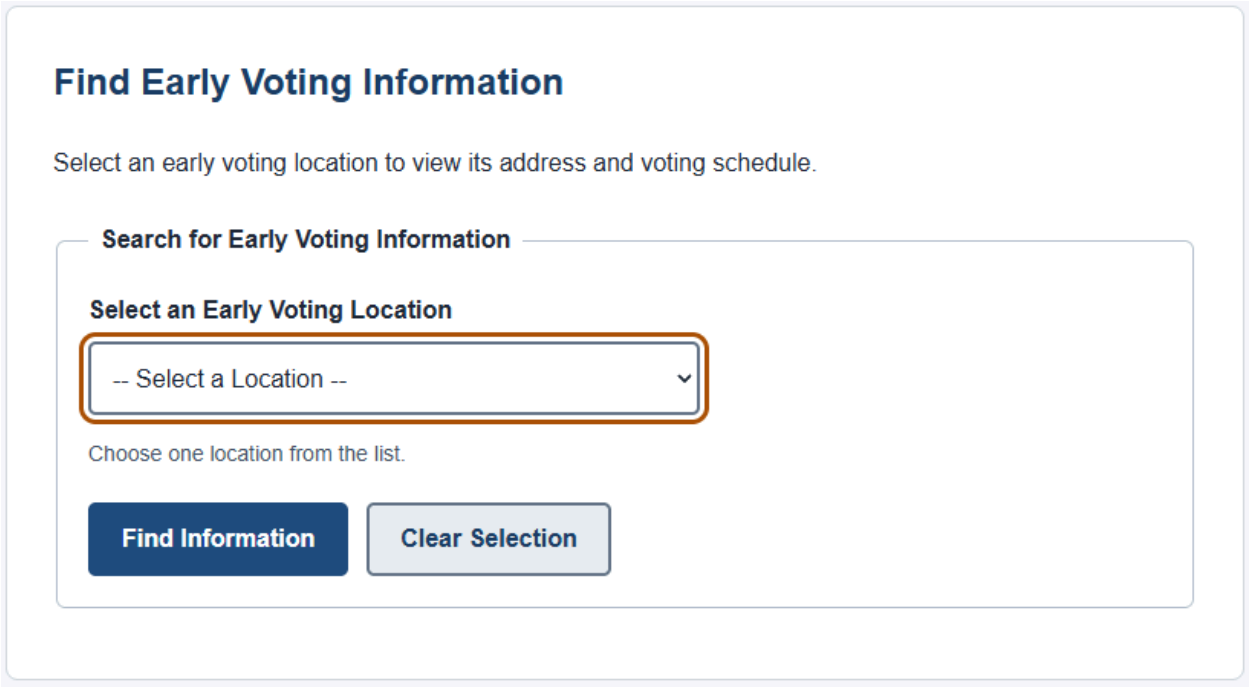



In this screenshot, the mouse is hovering over the “Find Information” button.

3. Focus State

The focus state is particularly important for users navigating the website with a keyboard rather than a mouse.

When users press the Tab key to navigate between the dropdown and buttons, the currently focused element receives a visible, high-contrast amber outline. This is implemented using the CSS :focus-visible pseudo-class with a 3-pixel outline and a 3-pixel offset. This allows keyboard users to clearly identify which element they are interacting with.



4. Active/Pressed State

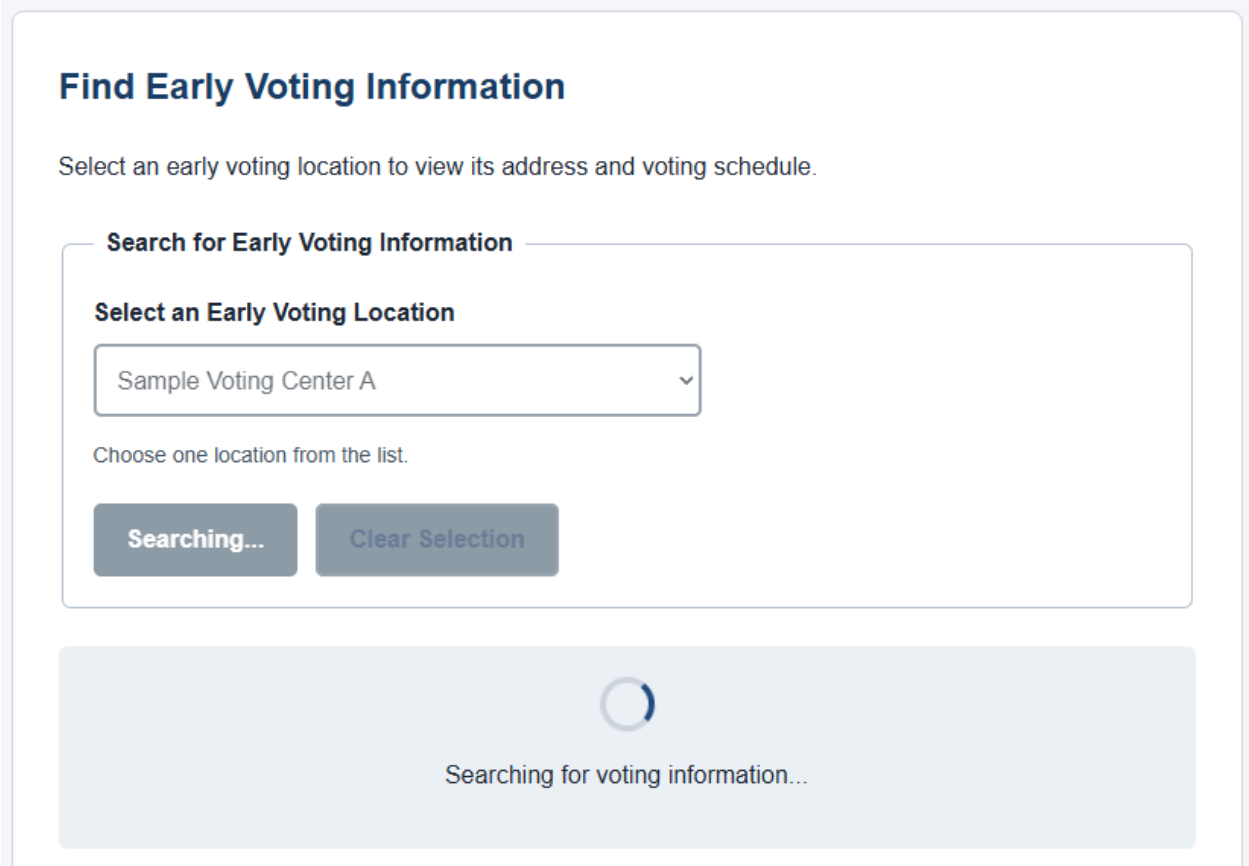
The active state provides immediate visual feedback when a user presses an interactive element. For example, when the "Find Information" button is pressed, it briefly scales down to 98% of its original size and changes to a darker blue. This is implemented using the CSS :active pseudo-class and a 100ms transform transition.

The dropdown uses the browser's native selection behavior when activated. These interactions help communicate that the user's input has been registered. It is hard to use a screenshot to represent this state, so please feel free to download the prototype from the “early-voting-prototype” folder found at the top level directory of this GitHub repository to try it yourself.

5. Disabled State

The disabled state occurs when the user submits a valid location selection and the interface begins processing the search. During this period, the dropdown, "Find Information" button, and "Clear Selection" button are temporarily disabled to prevent the user from changing their selection or submitting duplicate requests.

Disabled controls appear visually muted, and the search button's text changes to "Searching..." to explain why it is temporarily unavailable. JavaScript applies the disabled property to the controls and removes it once processing is complete.




6. Loading/System Processing State

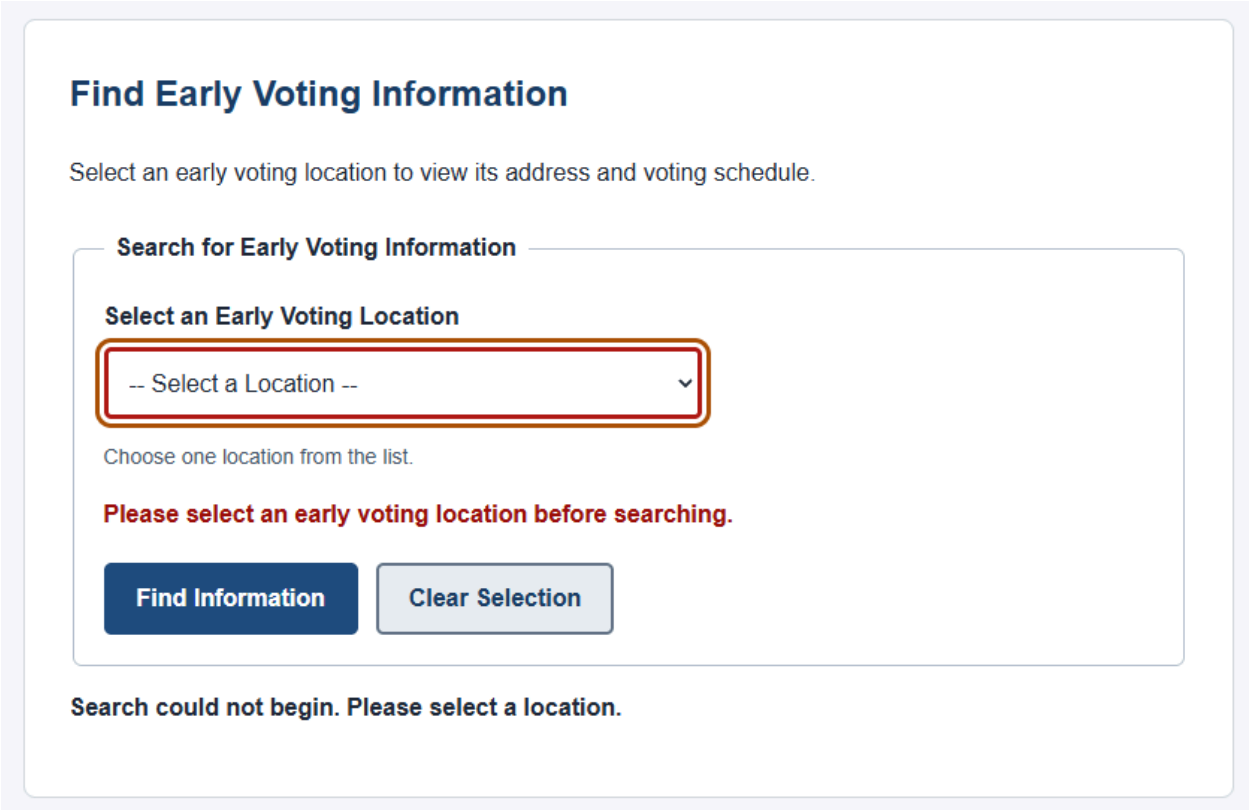
After a user selects a location and clicks "Find Information", the interface enters a loading state while the requested information is retrieved. For this prototype, I simulated an asynchronous request using JavaScript's setTimeout() function with an 800ms delay.

During this delay, a loading spinner appears alongside the message "Searching for voting information...". The search button also changes to "Searching...", and the form controls are temporarily disabled.

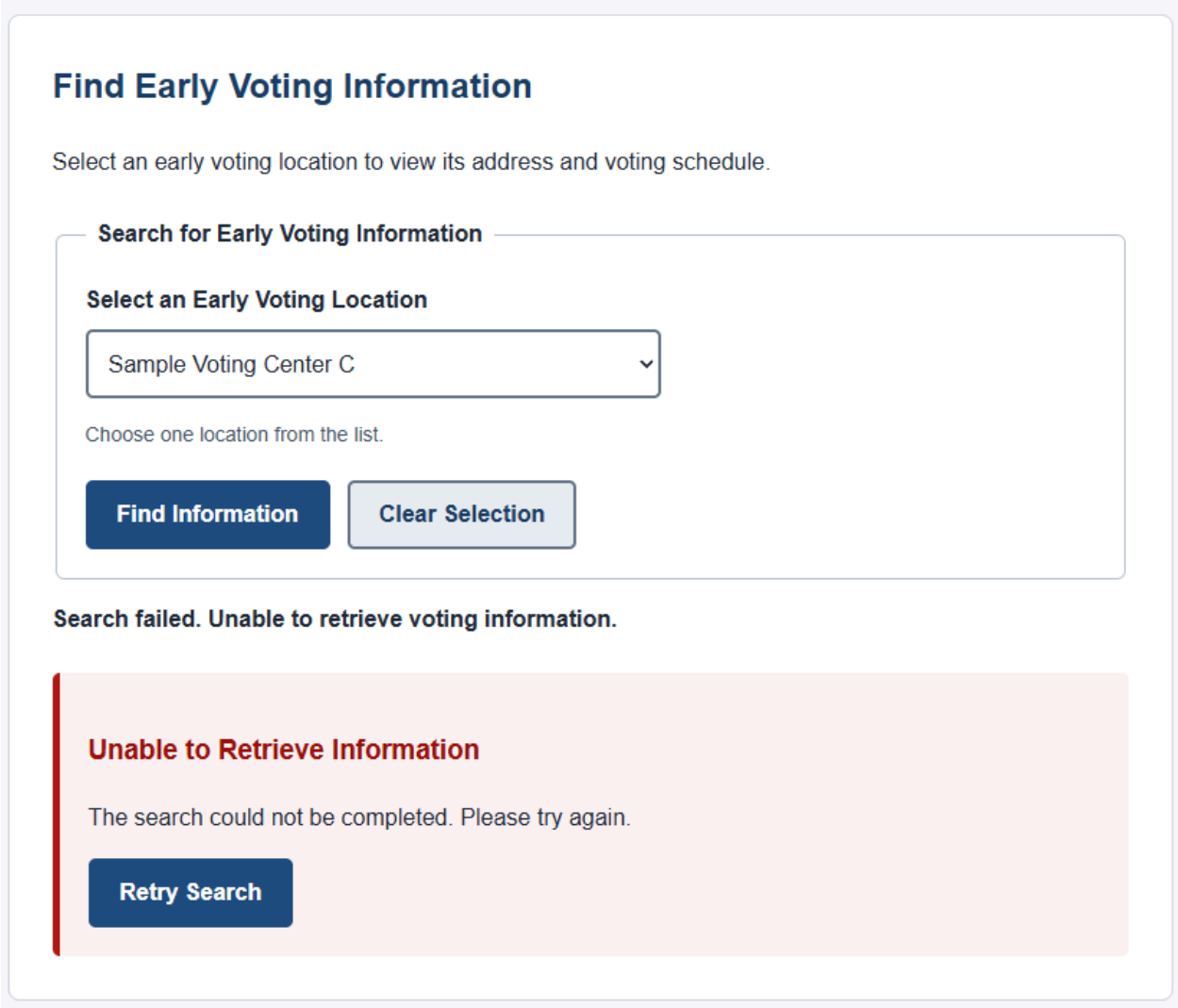
An ARIA live region announces the processing status to screen reader users. Once the simulated request finishes, the loading indicator disappears, the controls become available again, and either the results or an error message is displayed. This provides immediate feedback and prevents users from repeatedly submitting the same request because they are unsure whether the interface is responding.

7. Error/Validation Alert State
The prototype implements two different error scenarios.

The first occurs when a user clicks "Find Information" without selecting a location. The dropdown receives a red border, and an inline error message appears stating, "Please select an early voting location before searching." The interface also sets aria-invalid="true" on the dropdown and moves keyboard focus back to it, allowing users to identify and correct the problem.




The second error scenario simulates a failure to retrieve voting information from the server. For testing purposes, selecting "Sample Voting Center C" triggers this failure after the loading period. When this happens, the interface displays an error panel explaining that the search could not be completed, along with a "Retry Search" button. A screen reader status announcement also communicates the failure.




These two scenarios demonstrate how the interface handles both user input errors and asynchronous system failures while providing clear steps to proceed fluidly.

Overall State Transition Flows

The primary successful interaction follows this sequence: Default → Location Selected → Active → Loading/Disabled → Results Displayed → Default Controls Restored

If the user submits the form without selecting a location: Default → Active → Validation Error → Focus Returned to Dropdown → Corrected Selection → Loading → Results Displayed → Default Controls Restored

If the simulated server request fails (Voting Center C): Default → Location Selected → Loading/Disabled → Server Error → Retry Search or Select Another Location

## Step 3: Accessibility Architecture & WCAG Specification

**Semantic HTML Markup:** The prototype uses the following semantic elements:

| HTML Element | Purpose in the Prototype |
|---|---|
| <header> | Contains the Madison County Elections heading and page identification |
| <nav> | Provides navigation back to the main election website |
| <main> | Identifies the primary content of the webpage |
| <section> | Organizes the search interface, results, and error messages (when necessary) into distinct sections |
| <form> | Groups the location selection and search submission functionality |
| <fieldset> | Groups the related form controls together |
| <legend> | Provides a descriptive heading for the grouper form controls |
| <label> | Associates a descriptive label with the location dropdown |
| <button> | Provides native keyboard-accessible controls for searching, clearing selections, and retrying failed searches |

Using these elements rather than relying exclusively on generic  containers helps establish a logical document hierarchy and makes the interface easier to navigate with assistive technologies.

**ARIA & Screen Reader Hooks:**
In addition to semantic HTML, I implemented ARIA attributes to communicate information that may not be apparent to users relying on screen readers.

The following ARIA attributes are used throughout the prototype:

| ARIA Attribute | Implementation and Purpose |
|---|---|
| aria-label | Provides an accessible name for the breadcrumb navigation and other elements when necessary. |
| aria-labelledby | Associates the search, results, and error sections with their respective headings. |
| aria-describedby | Associates the dropdown with its help text and validation error message. |
| aria-invalid="true" | Indicates that the dropdown contains an invalid selection after an unsuccessful submission. |
| aria-live="polite" | Announces dynamic status changes without unnecessarily interrupting the screen reader. |
| aria-atomic="true" | Ensures the entire updated status message is announced. |
| aria-busy="true" | Indicates that the search form is currently processing a request. |
| aria-current="page" | Identifies Early Voting as the current page within the breadcrumb navigation. |

For example, when a user submits a search, the interface updates a live region with the message "Searching for early voting information." Once the search finishes, the region announces whether the voting information was successfully retrieved or whether an error occurred.

When the user submits the form without selecting a location, the dropdown receives aria-invalid="true" and displays an inline validation message. The message is programmatically associated with the dropdown using aria-describedby, allowing screen reader users to understand the problem and how to correct it.

Keyboard Navigation and Focus Management

As mentioned previously, the prototype is designed to be fully navigable without requiring a mouse. Users can press Tab to move forward through the interactive elements and Shift + Tab to move backward. The location dropdown supports selection through the arrow keys, while the search and reset buttons can be activated using Enter or Space.

When an invalid search is submitted, keyboard focus automatically returns to the location dropdown so the user can correct the problem. After a successful search or simulated server failure, focus moves to the corresponding results or error heading using tabindex="-1" and JavaScript's .focus() method.

This ensures that keyboard and screen reader users receive feedback and can continue interacting with the interface without becoming disoriented or stuck.

**Visual Accessibility:**
The visual design of the prototype considers color contrast, focus visibility, and interactive target sizing. The prototype uses dark text on light backgrounds, white text on dark blue buttons, and high-contrast borders around interactive controls.

Keyboard focus is indicated by a 3-pixel amber outline using CSS :focus-visible, helping users identify their current position within the interface. The dropdown and buttons are designed to be at least 48 pixels tall, exceeding the 44px minimum suggested. Consistent spacing between controls also helps reduce accidental selections on touchscreen devices.

These design choices aim to improve accessibility for users with visual, motor, and cognitive disabilities while also improving the general usability of the interface.

# Post-Mortem Reflection (Metacognition)

1. How did explicitly defining every component state (Hover, Focus, Active, Loading, Error) during the specification phase prevent dynamic usability bugs that static wireframes completely fail to address?

- Explicitly defining every component state during the specification phase helped me plan more in advance for what I wanted my eventual prototype to look like and account for how I wanted the interface to behave throughout the entire user interaction, rather than how I wanted it to look initially. I think a good example of this is the error state. Error handling is crucial to keeping a user from running into a brick wall with the website and also herding them towards solutions that will get them going again. Since I had to consider the error state from the beginning, I designed the HTML5 code around giving the user feedback when they chose the option that didn't work (Voting Center C). The website gives them a red box that says "Unable to Retrieve Information" and underneath that says "The search could not be completed. Please try again." The color and tenor of this message gives the user immediate negative feedback and prompts them to try again. If I had designed this from a wireframe, I likely would've never thought about the user getting to an error message because I would've been totally focused on what the initial state of the prototype looked like.

2. Describe a scenario in your prototype where system latency or asynchronous data processing required specialized micro-interaction feedback (e.g., loading skeletons or aria-live announcements). How does your design keep the user informed and prevent repeated submission errors?

- In my prototype, I am simulating system latency when the user clicks the "Find Information" button with a built-in 800ms delay to represent the time it may take for a real website to retrieve information from a database. During this time, it is put into the "Disabled" state, in which it grays out the two buttons to show that they are disabled, the text "Find Information" switches to "Searching…", and the dropdown menu is temporarily disabled while the search is "processing" as well. All of these factors combine to inform the user that their search is in fact being processed and to not tab away or close out the window. Disabling the controls also simultaneously prevents repeated submission errors.